# NovaTech Industries

## Checkpoint 2.6 — Step 3: Procurement Requests and Supplier Quotations

**Document type:** Technical learning guide and data-generation specification

**Version:** 1.0

**Date:** 21 October 2026

**Status:** Implementation exercise (synthetic data only)

*NovaTech Industries is a fictional company used for this educational case study. All records are synthetic.*

## 1. Objective

Extend the validated v2 reference data with realistic procurement transactions. Preserve the existing supplier, product, and supplier-product CSVs. Create six purchase requisitions and linked supplier quotations in the same v2 folder.

## 2. Business definitions

• **Purchase requisition:** An internal request to purchase a product, with a quantity and required delivery date.

• **Quotation:** A supplier's commercial offer in response to a requisition, with a validity period and currency.

• **Quotation item:** The quoted product, quantity, unit price, extra charges, and lead time.

• **Referential integrity:** A transaction's supplier, product, or requisition identifiers must exist in the appropriate parent dataset.

## 3. Input and output datasets

### *Inputs (already generated)*

File | Records | Purpose

suppliers.csv | 12 | Supplier master and status

products.csv | 20 | Product master

supplier_products.csv | 45 | Valid supplier-product relationships

### *New outputs*

File | Expected records | Important fields

purchase_requisitions.csv | 6 | requisition_id, product_id, requested_quantity, request_date, required_delivery_date, status

quotations.csv | 14 with seed 2027 | quotation_id, requisition_id, supplier_id, quotation_date, valid_until, currency, status

quotation_items.csv | 14 with seed 2027 | quotation_id, product_id, quoted_quantity, unit_price_eur, freight_eur, duties_eur, other_fees_eur, lead_time_days

The exact number of quotations is determined by the available supplier-product relationships. With the validated v2 reference generator and seed 2027, it is 14.

## 4. Business and data rules

1. Each requisition references an existing product.

2. Each quotation references an existing requisition and supplier.

3. Every quotation item references an existing quotation and the requisition's product.

4. A supplier may quote only products listed in supplier_products.csv.

5. Quoted quantity must meet or exceed requested quantity.

6. Unit price and lead time must be positive; monetary charges must be nonnegative.

7. All monetary fields are EUR in this introductory exercise; multi-currency conversion comes later.

8. An intentionally expired quotation is included to test future eligibility logic.

9. The script **does not** calculate supplier scores, approve orders, or make procurement recommendations.

10. The previously approved **manual review for insufficient performance history** remains in effect when we implement evaluation; performance scoring is not part of this step.

**Data modeling simplification:** Each requisition and quotation contains exactly one product line. Later we will support multiple items per requisition and quotation.

## 5. Implementation in VS Code

From your project root:

```
cd ~/Documents/Projects/ai-procurement-platform
```

Save the companion Python file as:

```
src/data_generation/generate_procurement_data_v2.py
```

Run:

```
python3 src/data_generation/generate_procurement_data_v2.py
```

**Expected output (using the existing v2 reference generator):**

```
Created purchase_requisitions.csv: 6 records
Created quotations.csv: 14 records
Created quotation_items.csv: 14 records
Validation passed: IDs, references, product matching, quantities, and prices.
Reproducible generation seed: 2027
Scenario: QUO001 has an expired validity date (intentional test case).
```

Inspect the generated files:

```
head -n 5 data/synthetic/v2/purchase_requisitions.csv
head -n 5 data/synthetic/v2/quotations.csv
head -n 5 data/synthetic/v2/quotation_items.csv
```

## 6. Understanding the code

• csv.DictReader loads CSV rows as dictionaries keyed by column names.

• random.Random(2027) produces reproducible sample values without changing global random state.

• Path(__file__).resolve().parents[2] locates the project root based on where the script is saved.

• datetime.date and timedelta generate and compare realistic dates.

• assert statements test expected relationships and numeric constraints before writing files.

• The generator reads existing v2 reference CSVs instead of recreating or overwriting them.

## 7. Worked purchasing-cost example

For one quotation, the estimated purchasing cost is:

total_cost_eur = quoted_quantity × unit_price_eur + freight_eur + duties_eur + other_fees_eur

For 20 units at EUR 75 each, plus EUR 100 freight and EUR 15 fees, the total is **EUR 1,615**. This is a cost illustration, not yet a supplier-selection result.

## 8. Validation and limitations

The generator verifies identifiers, relationships, product consistency, requested quantities, and positive prices/lead times. It intentionally creates QUO001 with an expired validity date. **Do not treat all generated quotations as eligible.** Eligibility filtering, delivery-date checks, historical quality and reliability metrics, and weighted scoring are separate upcoming steps.

The script uses Python's built-in random module for simulated data. It is appropriate for learning, not for financial or security-sensitive randomness. All records are artificial.

## 9. Completion checklist

■ The three original v2 CSVs remain unchanged.

■ The Python script is saved under src/data_generation/.

■ The script runs without errors.

■ Three new CSV files appear in data/synthetic/v2/.

■ The terminal reports successful validation.

■ The Markdown and PDF guides are saved under docs/data/.

## 10. Knowledge check

1. Why does a quotation reference a requisition ID?

2. Why should a quotation's supplier-product pair be checked against supplier_products.csv?

3. What is the difference between a valid database relationship and an eligible quotation?

## 11. Next checkpoint

**Checkpoint 2.6 — Step 4:** Load the six v2 datasets, apply eligibility rules, calculate comparable purchasing costs, and produce a traceable evaluation table. Weighted scoring will follow only after the required performance data and policy decisions are in place.